A Zero Trust Network with Automated Threat Detection and Response using Isolation Forest

Esborn Kihiu

Adm No. 168863

A Project Submitted to the School of Computing and Engineering Sciences, Strathmore University, for the Award of Bachelor of Science in Computer Networks and Cyber Security

STRATHMORE UNIVERSITY

NAIROBI, KENYA

October 2026

Declaration and Approval

I declare that this work has not been submitted or previously submitted and approved, in whole or in part, for the award of a degree by this or any other University. To the best of my knowledge and belief, the report contains no material previously published or written by another person except where due reference is made in the report itself.

Student

Esborn Kihiu

Adm No. 168863

Signature  …… Date ……

Supervisor

Mr. James Gikera

Lecturer, Computer Networks and Telecommunications

School of Computing and Engineering Sciences

Signature …… Date ……

Acknowledgement

I thank God for the strength and good health to complete this project. I am grateful to my supervisor, Mr. James Gikera, for his guidance and for the detailed feedback he gave on every draft, which greatly improved this work.

I also thank Strathmore University and the School of Computing and Engineering Sciences for the knowledge and resources that made this project possible. Finally, I thank my family and friends for their support and patience throughout my studies.

Abstract

Most enterprise networks still trust a device once it is inside the perimeter. If an attacker gets in, or a staff account is misused, there is little to stop them moving between internal systems, and signature-based tools often do not notice. This project designed and built a Zero Trust enterprise network with an automated threat detection and response layer based on the Isolation Forest algorithm. The network was first planned in Cisco Packet Tracer and was then rebuilt in Containerlab, where it runs real Arista cEOS switch software in containers on a single laptop. Users log in with IEEE 802.1X, and a FreeRADIUS server places each user in the VLAN for their role (admin, staff or guest). An access list on the core switch limits traffic between the segments, a Linux edge firewall separates the internal network from a DMZ web server and the internet, and VLAN 99 is used to quarantine devices. Traffic from the user ports is mirrored to a SIEM host, where a Python collector summarizes each host's activity every ten seconds into eight features, such as packet count, unique destination ports and failed logins. An Isolation Forest model, trained without labels, scores each window, and a simple policy rule flags windows with repeated failed logins. When a window is anomalous, a response engine classifies it and blocks the source IP for denial-of-service traffic, quarantines the host for port scans and brute-force attempts, disables the account for insider data theft, or raises an alert for off-hours activity. Scores, alerts and actions are shown on a Zabbix dashboard. The system is evaluated with simulated normal and attack traffic, using the detection rate, the false positive rate and the time from detection to action.

Keywords: *Zero Trust, 802.1X, Network Segmentation, Anomaly Detection, Isolation Forest, Automated Response, Containerlab*

Table of Contents

Declaration and Approval....ii
Acknowledgement....iii
Abstract....iv
Table of Contents....v
List of Figures....viii
List of Tables....ix
List of Abbreviations and Acronyms....x
Definition of Terms....xii
Chapter 1: Introduction....1
1.1 Background Information....1
1.2 Problem Statement....2
1.3 Objectives of the Study....2
1.3.1 General Objective....2
1.3.2 Specific Objectives....2
1.4 Scope and Limitations....3
1.4.1 Scope....3
1.4.2 Limitations....3
1.5 Significance of the Study....4
Chapter 2: Literature Review....5
2.1 Introduction....5
2.2 Security Challenges in Enterprise Networks....5
2.2.1 Limits of Perimeter-Based Security....5
2.2.2 Insider Threats and Credential Misuse....5
2.2.3 Limits of Signature-Based Detection....5
2.2.4 Slow Incident Response....6
2.2.5 Visibility and Alert Overload....6
2.3 Technology Review....6
2.3.1 Zero Trust Architecture....6
2.3.2 Authentication, Authorization and Accounting with 802.1X....7
2.3.3 VLAN Segmentation and Access Control Lists....7

2.3.4 Demilitarized Zone (DMZ)....8
2.3.5 Network Threats Considered....8
2.3.6 Machine Learning in Intrusion Detection....9
2.3.7 Isolation Forest Algorithm....9
2.3.8 Network Simulation and Emulation Platforms....11
2.3.9 Python Tools for Collection, Automation and Monitoring....12
2.4 Existing Solutions for Enterprise Threat Detection....12
2.5 Research Gap....13
2.6 Conceptual Framework....14
Chapter 3: Methodology....15
3.1 Introduction....15
3.2 Project Methodology....15
3.2.1 Requirements Gathering and Analysis....16
3.2.2 Quick Design....16
3.2.3 Prototype Development....17
3.2.4 Testing and Refinement....17
3.2.5 Final Validation....17
3.3 Change of Simulation Platform and Topology....17
3.4 System Requirements....19
3.4.1 Functional Requirements....19
3.4.2 Non-Functional Requirements....20
3.5 Implementation Requirements....21
3.5.1 Hardware Requirements....21
3.5.2 Software Requirements....21
3.6 Testing and Validation....22
3.6.1 Functional Testing....23
3.6.2 Security Testing....23
3.6.3 Data Collection and Labeling....24
3.6.4 Performance Testing....24
3.6.5 Evaluation Metrics....25
3.6.6 Validation of Results....26

Chapter 4: System Design.....27
4.1 Introduction.....27
4.2 Black-Box.....27
4.3 System Analysis.....28
4.3.1 Use-Case Diagram.....28
4.3.2 Sequence Diagram.....29
4.3.3 Activity Diagram.....32
4.3.4 Entity Relationship Diagram (ERD).....33
4.3.5 Class Diagram.....35
4.4 System Design.....36
4.4.1 System Architecture Diagram.....36
4.4.2 Network Topology.....38
4.4.3 Access Control Policy.....39
4.4.4 Detection Features and Response Rules.....40
References.....43

List of Figures

Figure 2.1: Isolating a normal and an anomalous session with random splits.....10
Figure 2.2: Conceptual framework.....14
Figure 3.1: Prototyping methodology and the prototypes built.....16
Figure 4.1: Black-box view of the system.....28
Figure 4.2: Use-case diagram.....29
Figure 4.3: Sequence diagram for one anomalous window.....31
Figure 4.4: Activity diagram of the detection and response loop.....33
Figure 4.5: Entity relationship diagram.....34
Figure 4.6: Class diagram.....36
Figure 4.7: System architecture.....37
Figure 4.8: Network topology as built in Containerlab.....38

List of Tables

Table 2.1: Threats considered and the traffic features they affect....8
Table 2.2: Comparison of network simulation and emulation platforms....11
Table 2.3: Comparison of existing threat detection approaches....13
Table 3.1: Packet Tracer design compared with the Containerlab build....18
Table 3.2: Functional requirements and how each is verified....20
Table 3.3: Hardware used....21
Table 3.4: Software used....22
Table 3.5: Attack scenarios used for testing....23
Table 3.6: Confusion matrix used for evaluation....25
Table 4.1: VLAN and addressing plan....39
Table 4.2: Entries of the ZT_POLICY access list....40
Table 4.3: Features extracted for each host in each window....41
Table 4.4: Response taken for each attack type....42

List of Abbreviations and Acronyms

AAA – Authentication, Authorization and Accounting

ACL – Access Control List

cEOS – Containerized Extensible Operating System (Arista)

CPU – Central Processing Unit

CSV – Comma-Separated Values

DHCP – Dynamic Host Configuration Protocol

DDoS – Distributed Denial of Service

DMZ – Demilitarized Zone

DoS – Denial of Service

FN – False Negative

FP – False Positive

HTTP – Hypertext Transfer Protocol

IDS – Intrusion Detection System

IEEE – Institute of Electrical and Electronics Engineers

IP – Internet Protocol

IPS – Intrusion Prevention System

ISE – Identity Services Engine

JSON – JavaScript Object Notation

LTS – Long-Term Support

MAC – Media Access Control

MSCHAPv2 – Microsoft Challenge Handshake Authentication Protocol version 2

MTU – Maximum Transmission Unit

NAS – Network Access Server

NAT – Network Address Translation

PEAP – Protected Extensible Authentication Protocol

RADIUS – Remote Authentication Dial-In User Service

RAM – Random Access Memory

SIEM – Security Information and Event Management

SNMP – Simple Network Management Protocol

SOAR – Security Orchestration, Automation and Response

SPAN – Switched Port Analyzer

SSH – Secure Shell

SVI – Switched Virtual Interface

SVM – Support Vector Machine

TCP – Transmission Control Protocol

TN – True Negative

TP – True Positive

UDP – User Datagram Protocol

UEBA – User and Entity Behavior Analytics

UML – Unified Modeling Language

VLAN – Virtual Local Area Network

ZTA – Zero Trust Architecture

Definition of Terms

Zero Trust – A security approach in which no user or device is trusted by default, even inside the network, and every request is checked before access is given.

802.1X – An IEEE standard for port-based access control, in which a switch port stays closed until the connected device authenticates.

Supplicant – The software on a user's device that sends the login details in 802.1X; wpa_supplicant in this project.

Authenticator – The switch that controls the port and passes the login request to the RADIUS server; acc1 in this project.

Dynamic VLAN assignment – Placing a user in a VLAN based on the reply from the RADIUS server instead of a fixed port setting.

Quarantine VLAN – A restricted VLAN (VLAN 99) for devices that have not logged in or have been flagged as suspicious.

Port mirroring (SPAN) – A switch feature that copies the traffic of chosen ports to another port for monitoring.

Session window – The activity of one host during one ten-second period; the unit that the model scores.

Anomaly score – A value from the Isolation Forest that shows how different a session window is from normal behavior.

Containerlab – An open-source tool that builds network labs from a topology file using containers.

Chapter 1: Introduction

1.1 Background Information

Enterprise networks today connect far more users, devices and services than they did a few years ago. Staff work remotely, applications have moved to the cloud and new devices join the network every day. Most of these networks are still protected with a perimeter model, where a firewall at the edge keeps outsiders out and anything already inside is trusted. According to Mensah (2024), this assumption is the main weakness of the perimeter model, because an attacker who gets past the edge, or an insider who misuses a valid account, can move between internal systems with very little resistance.

Zero Trust was proposed as a response to this problem. It works on the principle of “never trust, always verify”. No user or device is trusted because of where it is connected; every request is authenticated, authorized and monitored, and users get only the access their role needs (Aramide, 2024; Mensah, 2024). In a campus or enterprise network these ideas are usually put in place with IEEE 802.1X authentication against a RADIUS server, Virtual Local Area Networks (VLANs) that separate groups of users, and Access Control Lists (ACLs) that control the traffic between those groups. Commercial products such as Cisco Identity Services Engine (ISE) combine these into one identity-based policy system (Islam & Dhanekula, 2023).

Access control alone does not show what an authenticated user does after logging in. Many organizations still depend on signature-based intrusion detection, which can only recognize attacks that have been seen and described before. Unusual behavior such as a burst of requests, a scan of many ports or a large download at night does not match a stored signature, so it is often missed or noticed late (Rautaray, 2025). Machine learning offers a different approach, in which a model learns what normal traffic looks like and flags whatever does not fit (Hidayat et al., 2023).

Isolation Forest is an unsupervised anomaly detection algorithm that suits this task well. It does not need labeled attack data. It builds random trees and measures how quickly each record can be separated from the rest, and unusual records are separated in fewer steps, so they get a higher anomaly score (Fuhnwi et al., 2023; Lakshmi et al., 2024).

This project brings the two ideas together. A Zero Trust network was designed and built with 802.1X authentication, role-based VLANs, ACLs, a demilitarized zone (DMZ) and a quarantine

VLAN. Traffic from the user ports is copied to a monitoring host, where Python scripts extract features and an Isolation Forest model scores them. When the model flags a host, the system responds on its own by blocking the source IP address, moving the host to the quarantine VLAN, disabling the user's account or alerting the administrator. The network was first planned in Cisco Packet Tracer, but it was rebuilt in Containerlab with Arista cEOS switches so that the Python scripts could connect to real switch software, capture real traffic and change the configuration. The reasons for this change are explained in Section 3.3.

1.2 Problem Statement

Most enterprise networks still assume that a user or device that has passed the perimeter can be trusted. Once inside, an attacker, or a careless or malicious insider, can move from one segment to another, scan for services, flood a server or copy large amounts of data, often without being stopped. Firewalls and signature-based intrusion detection systems mainly look for known attacks, so new or slow behavioral attacks do not raise an alert (Rautaray, 2025). Even when an alert is raised, a person has to read it, investigate it and then change the network by hand, and Falowo and Bou Abdo (2026) note that this manual step is a major source of delay. During that time the attacker keeps working. The result is service disruption, data loss and access to systems that the user was never meant to reach. There is therefore a need for a network that verifies every user before giving access, limits each user to their own segment, watches behavior after login, and responds automatically when that behavior becomes abnormal.

1.3 Objectives of the Study

1.3.1 General Objective

The main objective of this project was to design and simulate a Zero Trust enterprise network with automated threat detection and response using the Isolation Forest machine learning algorithm.

1.3.2 Specific Objectives

i. To investigate Zero Trust architecture principles, identity-based access control and common threats to enterprise networks.

ii. To implement machine learning-based anomaly detection using the Isolation Forest algorithm and Python-based traffic and log collection.

iii. To design and develop a segmented Zero Trust network with VLANs, ACLs, a DMZ, a quarantine VLAN and automated threat response.

iv. To test and validate how well the system detects and responds to simulated attacks on the network.

1.4 Scope and Limitations

1.4.1 Scope

The project covered the design, build and testing of a Zero Trust enterprise network in a simulated environment, together with a detection and response layer. On the access control side it included 802.1X authentication of users against FreeRADIUS, dynamic VLAN assignment for admin, staff and guest users, an ACL on the core switch between the segments, a Linux edge firewall, a DMZ web server and a quarantine VLAN. On the detection side it included mirroring user traffic to a SIEM host, extracting eight features per host every ten seconds and scoring them with an Isolation Forest model, supported by a simple rule for repeated failed logins. The response side covered blocking a source IP address, quarantining a host, disabling an account and raising alerts, with all results shown on a Zabbix dashboard.

The network was built in Containerlab on a single laptop running Kali Linux, using Arista cEOS for the two switches and Debian-based Linux containers for the firewall, the servers and the user PCs. Python was used for traffic collection, feature extraction, the model and the automated response. The attacks studied were denial of service, port scanning, brute-force login attempts, insider data theft and off-hours activity.

1.4.2 Limitations

The project was carried out in a lab and not on a production network, so all user and attack traffic was generated by scripts. The model was therefore trained and tested on simulated traffic, which is simpler than real enterprise traffic. The laptop used has 8 GB of RAM, of which about 7.1 GB is usable, and each cEOS switch needs about 1.4 GB. Running three cEOS switches at once froze the machine, so the topology planned in Packet Tracer (two routers and four switches) was reduced to one core switch, one access switch and a Linux container acting as the edge router and firewall. As a result there is no redundancy and all users share one access switch.

The cEOS-lab software forwards traffic in software and does not accept ACLs on VLAN interfaces, so the segment policy was applied on the core switch's trunk port instead. Off-hours activity was simulated by marking time windows as night-time, because collecting real traffic at night would also have taught the model that night-time use is normal. FreeRADIUS was used instead of Cisco ISE; it applies the same AAA principles but does not have ISE's posture checks or graphical policy tools. Forensic investigation and cloud-native Zero Trust were outside the scope of the project.

1.5 Significance of the Study

The project shows, in a working lab, how Zero Trust access control and machine learning detection can support each other. Access control limits where a user can go, while the detection layer watches what the user does once allowed in. Because the model is unsupervised, it can flag behavior it was never trained on, which signature-based tools cannot do. Automating the response also shortens the time between detection and containment, which matters most for fast attacks such as denial of service (Jones, 2025).

The lab itself is a useful result. It runs on one laptop with free software, and the whole network can be rebuilt with one command. Students and small organizations that cannot afford Cisco ISE or a commercial SIEM can use the same approach to learn and test Zero Trust ideas. The design can also guide institutions such as universities, banks and SACCOs that want to add segmentation and behavior monitoring to their existing networks.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the literature and the technologies behind the project. It starts with the security problems that enterprise networks face, then looks at Zero Trust architecture and the technologies used to apply it: 802.1X with RADIUS, VLANs, ACLs and the DMZ. It then reviews machine learning for intrusion detection, with a focus on the Isolation Forest algorithm, and compares the platforms that can be used to simulate such a network. The chapter ends with a review of existing detection solutions, the gap that this project addresses and the conceptual framework.

2.2 Security Challenges in Enterprise Networks

Cloud services, remote work and the growing number of connected devices have made enterprise networks larger and harder to defend. Spending on security tools has gone up, but breaches still happen, and this has pushed organizations toward Zero Trust and toward better ways of detecting threats (Ahmadi, 2024). The main challenges are discussed below.

2.2.1 Limits of Perimeter-Based Security

Perimeter security trusts anything that is inside the network. Remote work and cloud services have made the line between inside and outside unclear, and an attacker who passes authentication can often reach internal systems with little resistance, because the model was not designed to keep checking users once they are in (Mensah, 2024).

2.2.2 Insider Threats and Credential Misuse

Insider threats are difficult to catch because the person uses valid credentials and normal access. A staff member copying large amounts of data, or an attacker using stolen credentials, does not trigger the usual alarms at login. Detecting this needs monitoring of what the access is used for, not only of whether it was granted (Aramide, 2024). Rauf et al. (2023), for example, detect insiders with a hybrid machine learning framework that learns from employee activity.

2.2.3 Limits of Signature-Based Detection

Signature-based tools compare traffic with patterns of known attacks. They work well against attacks that have been seen before, but new attacks, slow scans and changes in user behavior do

not match any rule (Rautaray, 2025). In many organizations such incidents are only discovered later, during an investigation.

2.2.4 Slow Incident Response

The time between detection and containment is valuable to an attacker. When every alert has to be read, escalated and acted on by a person, an attack can continue for minutes or hours. Studies of incident response show growing interest in automating at least the first containment steps, although trust in fully automatic actions is still limited (Falowo & Bou Abdo, 2026; Jones, 2025). Kumar et al. (2025) combined a Security Orchestration, Automation and Response (SOAR) platform with endpoint detection and response so that detected threats are handled automatically.

2.2.5 Visibility and Alert Overload

Enterprise networks produce more logs than analysts can review by hand. Important events are lost among routine ones, and slow attacks stay below the threshold of any single alert. Tools that summarize traffic and point out only the unusual parts are needed to make monitoring practical (Vadisetty & Polamarasetti, 2024).

2.3 Technology Review

The proposed system combines several technologies, each of which addresses part of the problems described in Section 2.2. They are reviewed in this section.

2.3.1 Zero Trust Architecture

Zero Trust Architecture (ZTA) removes the idea of a trusted internal network. Every request for a resource is authenticated and authorized wherever it comes from, and access is checked continuously instead of only at login (Al-Tamimi et al., 2024). Mensah (2024) summarizes its main principles as continuous verification, least-privilege access, segmentation of the network into small zones, and monitoring of behavior after access is granted. Least privilege means that a user or device gets only what its role needs, while segmentation means that a compromised device cannot move freely across the network. Chang and Xu (2026) show an access control model in which trust is checked again at each stage of a session instead of once at login, and Nam et al. (2026) review the methods used to turn observed behavior into a trust score.

In practice these principles are enforced by identity services, network segmentation and monitoring tools working together. Ahmadi (2025) describes identity-based segmentation that moves a device to a different segment automatically when its behavior becomes risky, which is close to the quarantine response used in this project. Zero Trust access control has also been applied outside office networks, for example in industrial networks (Abuhasel, 2023).

2.3.2 Authentication, Authorization and Accounting with 802.1X

Authentication, Authorization and Accounting (AAA) answers three questions about every connection: who the user is, what the user may do and what the user did (Aramide, 2024). In wired enterprise networks AAA is usually applied with IEEE 802.1X. The user's device, called the supplicant, sends its credentials through the switch, called the authenticator, to a RADIUS server, and the switch port stays closed until the server accepts the login.

The RADIUS server can also return the VLAN that the user should be placed in. This is called dynamic VLAN assignment, and it means that the VLAN follows the user's identity instead of the physical port. Cisco ISE is a widely used commercial implementation that adds device profiling and posture checks (Islam & Dhanekula, 2023). Bradatsch and Kargl (2024) describe how security service functions can be built into network-level access control, so that more than the login itself is considered when traffic is admitted. This project uses FreeRADIUS, an open-source RADIUS server that supports 802.1X with PEAP and MSCHAPv2 and can return a VLAN for each user (FreeRADIUS, n.d.). It applies the same AAA principles as ISE at no cost.

2.3.3 VLAN Segmentation and Access Control Lists

A VLAN divides one physical switched network into separate logical networks, each with its own broadcast domain. Devices in one VLAN can only reach another VLAN through a router or Layer 3 switch, where the traffic can be filtered (Baligodugula et al., 2024). ACLs are ordered lists of permit and deny rules applied to an interface. Used together, VLANs and ACLs make it possible to give each group of users only the access it needs (Bredesen & Mujeye, 2025). Bello et al. (2025) used dynamic microsegmentation together with role-based and attribute-based access control to apply Zero Trust to legacy systems.

Dube (2026) describes segmentation at several levels, from coarse network zones down to individual workloads. This project uses role-based VLANs for admin, staff and guest users, plus a

quarantine VLAN for devices that have not logged in or have been isolated. One ACL on the core switch controls the traffic between these VLANs.

2.3.4 Demilitarized Zone (DMZ)

A DMZ is a separate network segment for services that must be reachable from less trusted networks, such as a public web server. A firewall between the DMZ and the internal network makes sure that, even if a DMZ server is compromised, it still cannot open connections to internal systems. In this project the DMZ holds a web server that guests and staff can reach on port 80, while the edge firewall blocks any connection that the DMZ tries to open toward the internal VLANs.

2.3.5 Network Threats Considered

The project focuses on five threats that are common on enterprise networks and that change measurable features of the traffic. They are summarized in Table 2.1. Machine learning has already been used to detect several of these attacks from traffic features, for example port scans and DDoS attacks (Altidor & Talhi, 2024). Insider data theft and off-hours use are the hardest of these to catch, because the user has valid access and nothing looks wrong when they log in.

Threat	What happens	Feature most affected
Denial of service (DoS)	A host floods a server with requests to use up its resources	Packet count and request rate
Port scan	A host probes many ports on a target to find running services	Number of unique destination ports
Brute-force login	Repeated password guesses against the authentication service	Failed logins (RADIUS Access-Reject)
Insider data theft	A valid user downloads an unusually large amount of data	Bytes transferred
Off-hours use	A valid account is used at an unusual time, which may mean stolen credentials	Off-hours flag

Table 2.1: Threats considered and the traffic features they affect

2.3.6 Machine Learning in Intrusion Detection

Rule-based systems detect only what their rules describe, while machine learning learns patterns from data (Rautaray, 2025). Supervised methods such as random forests and neural networks learn from labeled examples of normal and attack traffic and usually reach high accuracy on known attack types (Ali et al., 2025; Hidayat et al., 2023). Their weakness is that they need labeled attack data, which is hard to get, and they may miss attacks that were not in the training set.

Unsupervised methods learn only what normal traffic looks like and flag anything that differs from it. They can detect new attacks, at the cost of more false alarms if the normal baseline is poor (Vadisetty & Polamarasetti, 2024). Kukartsev et al. (2024) show that simulated network traffic can be used to train and compare such models when real enterprise data is not available, which is the approach followed in this project. Syzranova et al. (2026) compared Isolation Forest, One-Class Support Vector Machine (SVM) and autoencoder detectors trained only on benign traffic. Both this setting and training on unlabeled mixed traffic were tested in this project, and the results are compared in Chapter 5.

2.3.7 Isolation Forest Algorithm

Isolation Forest is an unsupervised algorithm that finds anomalies by isolating them instead of by describing normal data. It builds many random trees. At each node it picks a random feature and a random split value between that feature’s minimum and maximum, and it keeps splitting until each record is on its own. Records that are very different from the rest end up alone after only a few splits, while normal records, which sit among many similar ones, need many splits (Fuhnwi et al., 2023). Figure 2.1 shows this with two features: the normal session in part (a) needed ten splits to be isolated, while the anomalous session in part (b) was isolated after two.



Figure 2.1: Isolating a normal and an anomalous session with random splits

The average number of splits needed across all the trees, called the path length h(x), is turned into an anomaly score using Equation 2.1:

$$s(x,\ n)\ =\ 2^{-E(h(x))\ /\ c(n)} \tag{2.1}$$

where E(h(x)) is the average path length of record x over all the trees and c(n) is the average path length for a sample of n records, which is used to normalize the score. Scores close to 1 point to anomalies, while scores well below 0.5 point to normal records (Lakshmi et al., 2024). The scikit-learn implementation used in this project reports a related decision score and allows the threshold to be tuned (scikit-learn developers, n.d.).

Isolation Forest has been applied to network intrusion detection in several recent studies. Fuhnwi et al. (2023) used it together with a One-Class SVM to detect intrusions in IP network traffic, Moomtaheen et al. (2024) tuned the Extended Isolation Forest to find malicious activity in Zeek connection logs, and Lakshmi et al. (2024) evaluated it on synthetic network traffic at different contamination rates, which is close to the simulated-traffic approach used in this project.

Isolation Forest suits this project for three reasons. It never needs labeled data: it can be trained on normal traffic alone, or on unlabeled traffic in which attacks are rare, so no labeled attack examples have to be prepared. It is fast to train and to score, which matters because every host is scored every ten seconds on a laptop with limited memory. It also works well with a small number of numeric features, such as the eight traffic features used here.

2.3.8 Network Simulation and Emulation Platforms

A network like this one can be studied with a simulator or with an emulator. Cisco Packet Tracer is a simulator that is widely used for teaching. It is easy to use and supports VLANs, ACLs and routing, but its devices are models of Cisco IOS rather than the real software, and scripts running on the host cannot log in to them the way they can with real devices (Cisco Networking Academy, n.d.). GNS3 runs real network operating system images inside virtual machines, which gives realistic behavior but uses a lot of memory for each device. Containerlab builds a lab from a short topology file and runs each network operating system as a container (Containerlab, n.d.). Arista's cEOS-lab is the EOS switch software packaged for this purpose; it supports 802.1X, VLANs, ACLs, port mirroring and SSH, and it is free to download (Arista Networks, n.d.). Table 2.2 compares the three options against the needs of this project.

Container-based emulation runs unmodified software in lightweight containers, but its timing can become less accurate when the containers ask for more CPU or input/output than the machine can give (Chen et al., 2025). This was a real concern on an 8 GB laptop, and it is one reason why the lab is deployed one node at a time and Zabbix is stopped during data collection.

Criterion	**Cisco Packet Tracer**	**GNS3**	**Containerlab with cEOS**
Device software	Simulated models of Cisco IOS	Real images in virtual machines	Real Arista EOS in containers
Scripts on the host can log in (SSH)	No	Yes	Yes
Real packets available to Python	No	Yes	Yes (tcpdump in the containers)
Memory per switch	Very low	High (one virtual machine each)	About 1.4 GB per cEOS node
Rebuilding the lab	Open a saved .pkt file	Saved project	One topology file and one script
Cost	Free	Free; some images need licences	Free

Table 2.2: Comparison of network simulation and emulation platforms

Containerlab with cEOS was chosen because it was the only option that met all the needs of the project: real switch software, 802.1X with dynamic VLANs, SSH access for the automated response and real packets for the detection pipeline, while still fitting on an 8 GB laptop.

2.3.9 Python Tools for Collection, Automation and Monitoring

Python was used for every software part of the project. Packets mirrored to the SIEM host are captured with tcpdump and read by a small Python reader built on the struct module, which takes only the header fields needed for the features. Scapy was tried first but used too much memory on large captures. Netmiko, a Python library for connecting to network devices over SSH, is used to push configuration to the switches and is also used by the response engine (Byers, n.d.; Choi, 2024). Scikit-learn provides the Isolation Forest and the feature scaler (scikit-learn developers, n.d.), while pandas and NumPy are used to prepare the data.

Zabbix is an open-source monitoring platform. In this project it polls the two switches over SNMP, receives the values that the Python scripts push to it, shows them on dashboards and opens a problem whenever a trigger fires (Zabbix, n.d.).

2.4 Existing Solutions for Enterprise Threat Detection

Enterprise security usually relies on several tools used together. Firewalls enforce access rules at network boundaries. Intrusion detection systems (IDS) such as Snort and Suricata compare traffic with signatures and raise alerts, and intrusion prevention systems (IPS) also block the matching traffic. Security Information and Event Management (SIEM) platforms collect logs from many devices in one place, and User and Entity Behavior Analytics (UEBA) tools add machine learning to spot unusual user behavior. Identity-based access control platforms such as Cisco ISE apply policy at the point where users connect (Islam & Dhanekula, 2023). Table 2.3 compares these approaches with the proposed system.

Approach	Detection basis	New attacks	Response	Main limitation
Firewall	Fixed rules on addresses and ports	No	Blocks by rule	Does not look at behavior
Signature IDS / IPS	Known attack signatures	No	Alert (IDS) or block (IPS)	Misses unknown attacks
SIEM	Log correlation rules	Partly	Alert to an analyst	Response is manual
UEBA	Behavior baselines (machine learning)	Yes	Alert to an analyst	Costly; not linked to network control
Cisco ISE	Identity and device posture at login	No	Assigns VLAN or policy	Little view of behavior after login
This project	Isolation Forest on traffic features	Yes	Automatic block, quarantine or alert	Tested on simulated traffic only

Table 2.3: Comparison of existing threat detection approaches

Each of these tools covers part of the problem. Most depend on known signatures or on a person to decide what to do. Those that use behavior analytics are often expensive and are not linked to the access control layer, so a suspicious host is reported but not contained.

2.5 Research Gap

The literature shows that Zero Trust access control and machine learning detection are both well studied, but they are usually treated separately. Zero Trust work focuses on identity, policy and segmentation (Ahmadi, 2024; Mensah, 2024), while studies of machine learning intrusion detection focus on accuracy on public datasets (Ali et al., 2025; Hidayat et al., 2023) and rarely act on the network. Studies of automated response show that organizations are interested in automation, but few working examples link detection directly to network controls (Falowo & Bou Abdo, 2026). There is also little work that students or small organizations can reproduce without expensive licences. This project addresses these gaps by building a reproducible lab in which an unsupervised detector watches the traffic of a segmented Zero Trust network, and its output changes the network directly through ACL entries and VLAN quarantine.

2.6 Conceptual Framework

Figure 2.2 shows the conceptual framework of the study. The independent variables are the three parts of the system: Zero Trust access control, anomaly detection and automated response. Access control decides who gets onto the network and which segment they can reach. Anomaly detection watches what each host does after login and gives every ten-second window an anomaly score. Automated response turns a high score into an action suited to the type of attack.

The dependent variable is the security of the network. It is measured by how many attacks are detected, how many normal windows are wrongly flagged, how long it takes from detection to action, and whether a compromised host is stopped from reaching other segments. The moderating variables affect how well the independent variables work: the quality of the traffic used to train the model, the threshold chosen for the anomaly score and the limited memory of the lab machine.

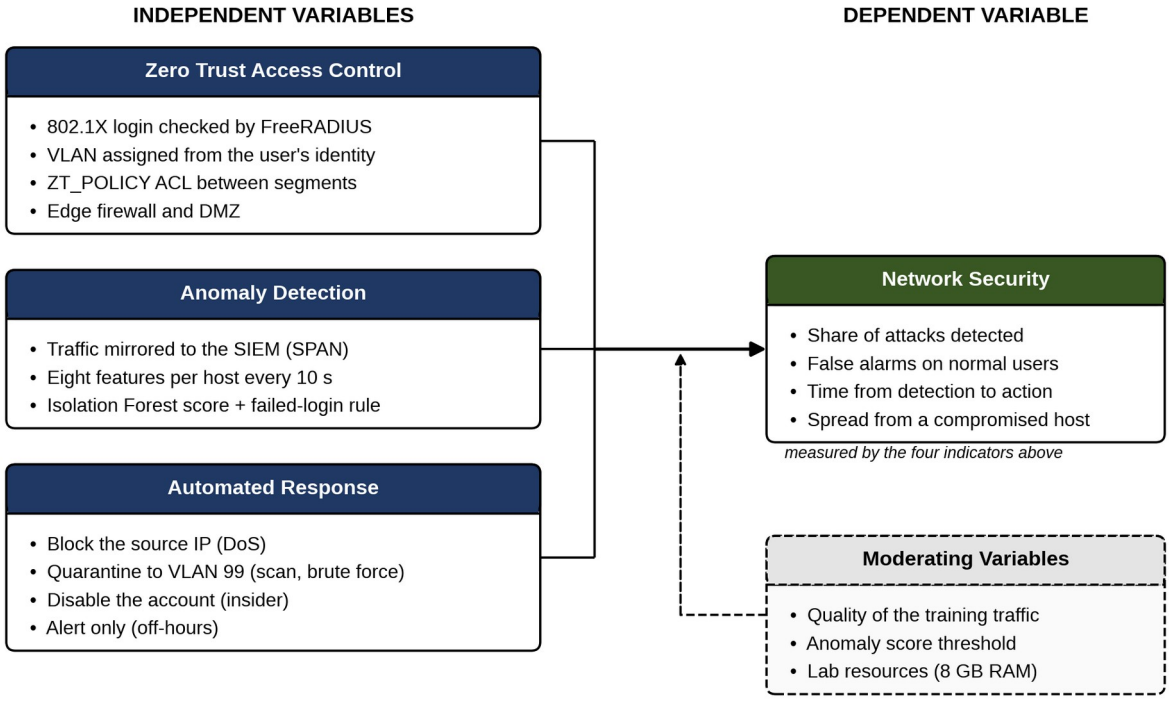


Figure 2.2: Conceptual framework

Chapter 3: Methodology

3.1 Introduction

This chapter describes how the project was carried out. It explains the development methodology, the move from Cisco Packet Tracer to Containerlab and the changes this made to the topology, the functional and non-functional requirements, the hardware and software used, and the way the system is tested and validated.

3.2 Project Methodology

The project used a quantitative approach, because the success of the system is measured with numbers taken from simulated attack runs: the detection rate, the false positive rate and the response time. Prototyping was chosen as the development methodology. In prototyping, a working version is built early, tested, and improved in repeated cycles until it meets the requirements.

This fitted the project for two reasons. First, several parts carried technical risk, such as whether 802.1X with dynamic VLANs would work on cEOS at all, and building a small prototype first was the quickest way to find out. Second, many problems only showed up once the lab was running, so each round of testing changed the design. Figure 3.1 shows the cycle and the three prototypes built in this project.

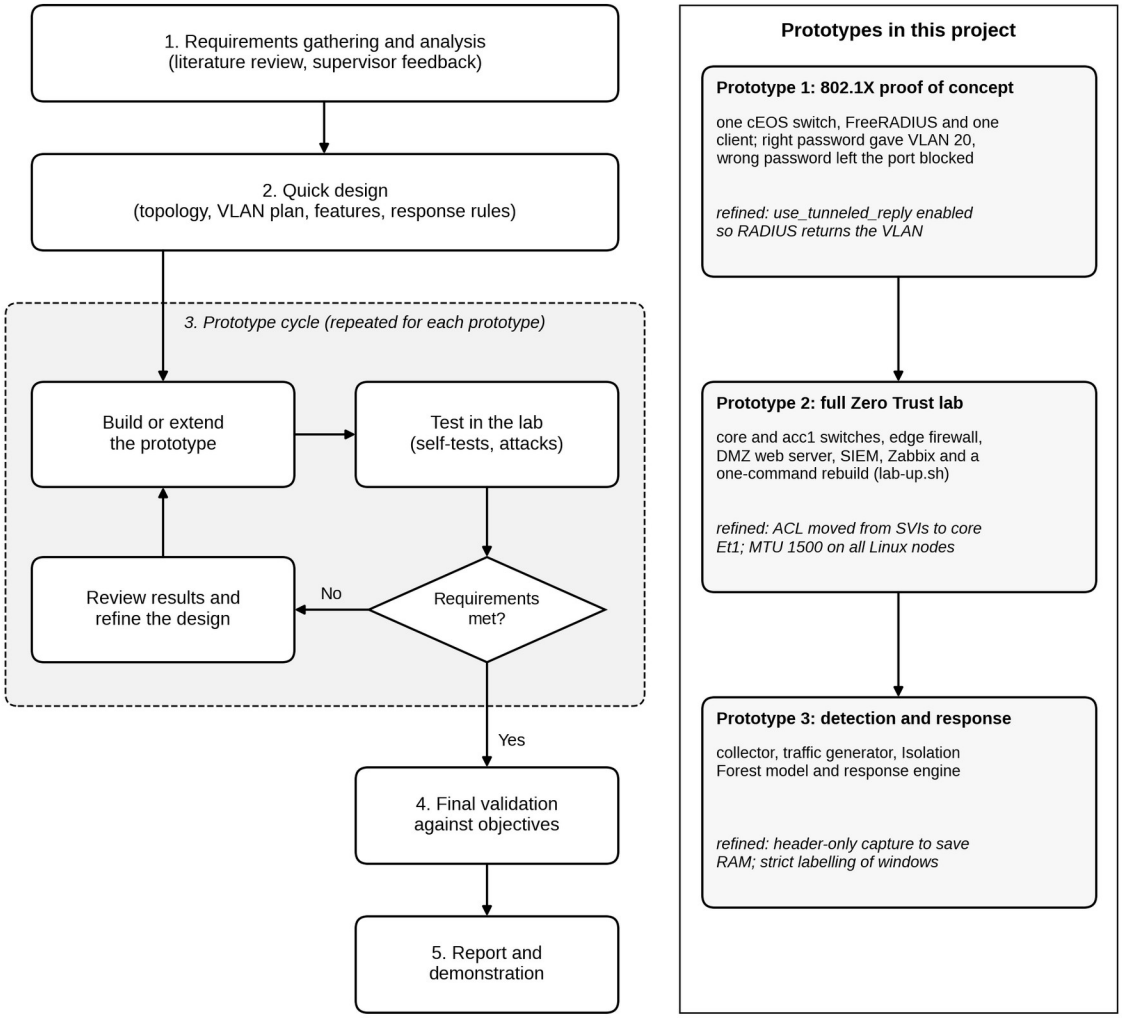


Figure 3.1: Prototyping methodology and the prototypes built

3.2.1 Requirements Gathering and Analysis

Requirements were taken from the literature review in Chapter 2, from the objectives in Section 1.3 and from the supervisor's feedback on earlier drafts. They were grouped into functional and non-functional requirements, which are described in Section 3.4.

3.2.2 Quick Design

A first design was drawn in Cisco Packet Tracer with VLANs, ACLs, a DMZ and a quarantine VLAN. It was then turned into a Containerlab topology file, an addressing plan, a list of the features the model would use and the rules the response engine would follow.

3.2.3 Prototype Development

Three prototypes were built, each one adding to the one before. Prototype 1 was a proof of concept with one cEOS switch, a FreeRADIUS server and one client, built to test the riskiest assumption first. With the correct password the client was placed in VLAN 20, and with a wrong password the port stayed blocked. Prototype 2 was the full Zero Trust network: the core and access switches, the edge firewall, the DMZ web server, the SIEM host and Zabbix monitoring, all rebuilt by one script (lab-up.sh). Prototype 3 adds the detection and response layer, made up of the collector, a traffic generator for normal and attack traffic, the Isolation Forest model and the response engine.

3.2.4 Testing and Refinement

Each prototype was tested before the next one was started, and the problems found during testing changed the design. For example, FreeRADIUS did not return the VLAN until the use_tunneled_reply option was enabled. cEOS rejected ACLs on VLAN interfaces, so the policy was moved to the core switch's trunk port. The first full dataset was also discarded after large downloads kept failing; the cause was that the Linux containers used a 9,500-byte MTU while the switches routed at 1,500 bytes, and all lab interfaces were then set to 1,500.

3.2.5 Final Validation

The finished system is validated against the objectives using the tests and metrics described in Section 3.6, and the results will be reported in Chapter 5.

3.3 Change of Simulation Platform and Topology

The proposal planned to build the network in Cisco Packet Tracer and GNS3, with Cisco ISE for authentication. During the first phase of the project this plan was changed, for three reasons. First, the detection and response layer needs Python scripts on the host that can log in to the switches, capture packets and change the configuration. Packet Tracer devices cannot be reached by scripts on the host in this way, so the automated response could not have been shown working on the network. Second, Cisco ISE needs a licence and far more memory than the lab machine has. Third, GNS3 runs each device as a virtual machine, which would not fit in 8 GB of RAM together with the servers and the monitoring tools.

The network was therefore rebuilt in Containerlab, with Arista cEOS-lab 4.36.2F for the switches and FreeRADIUS for AAA. The security design stayed the same: users still authenticate with

802.1X, get a VLAN based on their identity, are limited by ACLs and are separated from the DMZ by a firewall. The topology, however, had to be reduced. The first deployment with three cEOS switches froze the laptop, so the two routers and four switches of the Packet Tracer design became one core switch, one access switch and a Linux container that does the work of the edge router. Table 3.1 compares the two designs.

Element	**Packet Tracer design (proposal)**	**Containerlab build (current)**
Platform	Cisco Packet Tracer and GNS3	Containerlab 0.81.0 on Kali Linux
Internet / ISP	ISP-SIM-RTR (Cisco 2911 router)	The host's Docker network with NAT
Edge router and firewall	ZeroTrust-Edge-RTR with WAN_INBOUND_ACL	edge: Linux container with iptables
Core switch	ZeroTrust-Core-SW (Cisco 3560)	core: Arista cEOS
Access switches	Three Cisco 2960 switches	One switch, acc1: Arista cEOS
AAA server	Cisco ISE	FreeRADIUS 3.2.1
Segment policy	GUEST_ACL and QUARANTINE_ACL on the SVIs	One ACL, ZT_POLICY, inbound on core Ethernet1
Traffic collection	Simulated log collector	SPAN copy of Et2 to Et4 sent to the SIEM host
Monitoring	Not planned	Zabbix 7.0
Network devices	2 routers and 4 switches	2 switches and 1 Linux firewall

Table 3.1: Packet Tracer design compared with the Containerlab build

The reduced topology keeps every security function of the original design. The three access switches in the Packet Tracer design mainly separated the users physically. In a Zero Trust design the separation comes from the VLAN given at login and not from the switch a user is plugged into, so one access switch is enough. The edge container provides routing, NAT and the perimeter firewall while using less than 1 MB of memory, compared with about 1.4 GB for each cEOS switch.

3.4 System Requirements

3.4.1 Functional Requirements

The functional requirements describe what the system must do. They came from the objectives and from the problems identified in Chapter 2.

Access control. Every user port on the access switch must stay closed until the user logs in with 802.1X. FreeRADIUS must check the username and password and return the user's VLAN, so that the admin user (bob) is placed in VLAN 10, the staff user (alice) in VLAN 20 and the guest user (guest1) in VLAN 30. A wrong password must leave the port blocked, and ports that have not authenticated stay in the quarantine VLAN 99.

Segmentation. Guests must not reach the admin, staff or quarantine subnets. A quarantined device may reach only the RADIUS server on UDP port 1812, so that it can log in again, and the SIEM host, so that it can be investigated. The internal VLANs may reach the DMZ web server only on TCP port 80, and the DMZ must not be able to open connections into the internal network.

Collection and detection. The SIEM host must receive a copy of all traffic on the user ports and turn it into one record per host every ten seconds, with eight features. Failed logins must be read from the FreeRADIUS log and counted against the host that caused them. The Isolation Forest model must be trained without labels, must give each record an anomaly score and must flag records above a threshold tuned on normal traffic. A window with five or more failed logins must also be flagged, because a slow password attack changes no other feature.

Response and monitoring. For a flagged record the system must work out the likely attack type and act without waiting for a person: block the source IP address for denial of service, move the host to VLAN 99 for a port scan or a brute-force attempt, disable the user's account for insider data theft, and raise an alert only for off-hours activity or anomalies it cannot classify. Every action must be logged with its time, and the administrator must be able to release a blocked or quarantined host. Anomaly scores, alerts and actions must appear on a Zabbix dashboard together with the health of every node. Table 3.2 lists these requirements with the part of the lab that meets each one and how each one is checked.

No.	Requirement	Met by	How it is verified
FR1	Users log in with 802.1X before getting network access	acc1 Et2 to Et4 and FreeRADIUS	show dot1x hosts; wrong-password test
FR2	The VLAN is assigned from the user’s identity	FreeRADIUS users file	bob in VLAN 10, alice in 20, guest1 in 30
FR3	Guests cannot reach admin, staff or quarantine subnets	ZT_POLICY on core Et1	Ping tests and ACL hit counters
FR4	The DMZ cannot start connections to the internal network	edge iptables rules	Ping from web to RADIUS fails; DROP counters
FR5	Traffic is summarized per host every 10 seconds	SPAN to siem; zt_collector.py	Offline tests; dataset files
FR6	Each window is scored by the Isolation Forest and the failed-login rule	zt_model.py	5-fold leave-one-run-out evaluation
FR7	Anomalies are answered automatically by attack type	zt_respond.py	Deny entry, VLAN 99 or failed login seen after each attack
FR8	Scores, alerts and actions reach the administrator	Zabbix and zt_zbx_push.py	Dashboard pages and Problems list

Table 3.2: Functional requirements and how each is verified

3.4.2 Non-Functional Requirements

Performance. Each ten-second window must be captured, scored and, if needed, acted on before the next window ends, so the time from detection to action should be well under ten seconds. Normal users must not notice the monitoring, because it works on a mirrored copy of the traffic and not in the traffic path.

Accuracy. The detector should flag most attack windows while keeping false alarms on normal users low. A wrong quarantine cuts off a legitimate user, so the threshold is tuned on a separate normal run, and off-hours anomalies only raise an alert.

Reliability and reproducibility. The whole lab must be rebuilt from scripts with one command (lab-up.sh), and it must check itself after every build. Six self-tests confirm that the guest can reach the DMZ web server and the internet but not the RADIUS server or the staff PC, that the staff PC can reach the RADIUS server, and that the DMZ cannot reach the RADIUS server.

Resource use. The lab must run on a laptop with 8 GB of RAM. lab-up.sh refuses to start if less than 4.5 GB is free and deploys one node at a time, and Zabbix is stopped while datasets are collected. The collector keeps only the first 128 bytes of each packet to save memory.

Safety and maintainability. The traffic generator must never send traffic outside the lab, so its agent refuses any target outside 10.10.0.0/16. The system must use free and widely available tools, and every configuration file and script is kept in Git so that the lab can be changed and rebuilt easily. Lab passwords exist only in the configuration files and must be removed before the code is shared.

3.5 Implementation Requirements

3.5.1 Hardware Requirements

The project was built and tested on one laptop, described in Table 3.3. Memory was the main constraint. The full lab with Zabbix uses about 5.5 GB of the 7.1 GB available, which is why the topology was reduced and why lab-up.sh checks the free memory before it starts.

Component	Specification
Processor	AMD Ryzen 5 5600U, 6 cores and 12 threads
Memory	8 GB DDR4 (about 7.1 GB usable)
Storage	NVMe SSD with a 221 GB system partition
Operating system	Kali Linux Rolling, 64-bit, installed directly on the laptop
Network	Internet connection for downloads and for the guest internet test

Table 3.3: Hardware used

3.5.2 Software Requirements

All the software used is free. Table 3.4 lists each package with the version used and its purpose in the project.

Software	Version	Purpose
Kali Linux	Rolling	Host operating system
Docker	28.5.2	Runs all the lab containers
Containerlab	0.81.0	Builds the lab from zt-dev.clab.yml
Arista cEOS-lab	4.36.2F	Software for the core and access switches
Debian (zt-node image)	12 (bookworm)	Base image for the edge, servers and PCs
FreeRADIUS	3.2.1	AAA server for 802.1X
wpa_supplicant	2.10	802.1X client on the three PCs
iptables	Debian 12 package	Edge firewall and NAT
tcpdump	4.99.3	Packet capture on the SIEM host
Python	3.14 (host), 3.11 (containers)	Collector, traffic generator, model and response
Netmiko	4.8.0	SSH to the switches for configuration and response
scikit-learn, pandas, NumPy	Current stable releases	Isolation Forest, scaling and data handling
Zabbix	7.0 LTS	Monitoring dashboards and alerts
Git	Current	Version control and automatic backups
Visual Studio Code	Current	Writing and editing scripts
Cisco Packet Tracer	8.x	Initial design only (Section 3.3)

Table 3.4: Software used

3.6 Testing and Validation

Testing checks each part of the system on its own first, and then the whole system against the objectives.

3.6.1 Functional Testing

The access control layer is tested with the self-tests in lab-up.sh and with manual checks. The command show dot1x hosts on acc1 must show all three users authenticated and in the right VLANs, and a login with a wrong password must be rejected with the port left blocked. Ping and web tests between the segments are compared with the expected results in Table 3.2, and the hit counters on ZT_POLICY and on the edge firewall confirm which rule handled each packet.

The collector and the traffic generator have offline tests (test_collector.py with ten checks and test_traffic.py with six) that run on saved data without the lab, so they can be repeated after every change. The model is tested with the leave-one-run-out method described in Section 3.6.3, so it is always scored on runs it has never seen.

3.6.2 Security Testing

Attack traffic is generated by zt_traffic.py, which copies a small agent into each PC container and controls it from the host. Normal background traffic, made up of web page loads, file downloads and pings, runs on all three PCs throughout every test, so the detector has to find the attacker among normal users. Table 3.5 lists the attack scenarios and the response expected for each one.

Attack	**Source**	**How it is generated**	**Intensities**	**Expected response**
Denial of service	guest-pc	HTTP flood on the DMZ web server	40 to 120 requests/s	Block source IP
Port scan	guest-pc	TCP connect scan of the web server	10 to 50 ports/s	Quarantine host
Brute force	guest-pc (injected at RADIUS)	Wrong passwords for bob, alice and admin	1 to 8 attempts/s	Quarantine host
Insider data theft	staff-pc	Repeated download of a 20 MB file	0.5 to 2.5 MB/s	Disable account
Off-hours use	staff-pc	Normal traffic in windows marked as night	Not applicable	Alert only

Table 3.5: Attack scenarios used for testing

802.1X login frames never reach the IP network, so a password attack is only visible as the RADIUS requests that the switch relays and the rejects that follow. A real 802.1X client also retries only about every 20 seconds, which is too slow to look like an attack. The generator therefore sends the relayed requests straight to FreeRADIUS, using the switch's NAS address (10.10.10.11) and guest-pc's MAC address. It cannot run on the SIEM host, because the SIEM's switch port is the SPAN destination, which only receives traffic.

3.6.3 Data Collection and Labeling

The datasets were collected with collect_datasets.sh. Four normal runs of ten minutes each were captured: normal_01 and normal_02 are used for training, normal_03 to tune the threshold and normal_04 as unseen normal test data. The threshold is the 99th percentile of the scores on normal_03, which aims at about 1% false alarms. Each attack was then run five times for 60 seconds, once at each of the intensities in Table 3.5, with the attack active from the 10th to the 50th second.

The model is evaluated with 5-fold leave-one-run-out cross-validation. In fold k, run k of every attack type is held out for testing and the model is trained on two other runs of each type, so every run, including the weakest attacks, is tested by a model that never saw it. Two training modes are compared: normal-only, where the model sees only normal traffic, and mixed, where it sees unlabeled traffic in which attacks make up about 9% of the windows.

Every ten-second window is labeled as normal or attack only so that the results can be scored; the model never sees the labels. A window is labeled as an attack only if the attack covered at least 90% of it. Windows that the attack only partly covered are left out, so that they are not counted as false alarms or missed attacks. Because FreeRADIUS delays each reject by one second, the window just after a brute-force run is also left out.

3.6.4 Performance Testing

Performance testing measures the time from the end of a window to the completed response (Equation 3.6), using timestamps written by the pipeline at each step. It also checks that CPU and memory use on each node stay within limits during attacks. Every node pushes its CPU, memory and packet rates to Zabbix every 30 seconds, so the load on the lab can be compared between normal and attack runs.

3.6.5 Evaluation Metrics

The detector's output is compared with the labels using the four counts of a confusion matrix, shown in Table 3.6. A true positive (TP) is an attack window that was flagged, a false negative (FN) is an attack window that was missed, a false positive (FP) is a normal window that was flagged and a true negative (TN) is a normal window that was not flagged.

Actual class	**Predicted attack**	**Predicted normal**
Attack window	True positive (TP)	False negative (FN)
Normal window	False positive (FP)	True negative (TN)

Table 3.6: Confusion matrix used for evaluation

From these counts the detection rate (DR, also called recall), the false positive rate (FPR), the precision (P), the accuracy (Acc) and the F1-score are calculated for each attack type and each intensity using Equations 3.1 to 3.5. The F1-score balances precision against the detection rate, which matters here because the test runs contain many more normal windows than attack windows.

$$DR = \frac{TP}{TP + FN} \tag{3.1}$$

$$FPR = \frac{FP}{FP + TN} \tag{3.2}$$

$$P = \frac{TP}{TP + FP} \tag{3.3}$$

$$Acc = \frac{TP + TN}{TP + TN + FP + FN} \tag{3.4}$$

$$F1 = \frac{2 \times P \times DR}{P + DR} \tag{3.5}$$

The response time of each incident is measured from the moment the ten-second window closes to the moment the response action is confirmed on the network, as in Equation 3.6.

$$T_r = t_{action} - t_{window} \tag{3.6}$$

3.6.6 Validation of Results

The system will be considered valid if legitimate users in all three VLANs keep working without being flagged, each attack type is detected at its stronger intensities, DoS sources are blocked, scanning and brute-force hosts end up in VLAN 99, the account is disabled after insider data theft, off-hours activity raises an alert without cutting access, guests never reach internal resources, and every action appears in Zabbix with its timing. Each of these checks maps back to one of the specific objectives in Section 1.3.2.

Chapter 4: System Design

4.1 Introduction

This chapter presents the design of the system. It starts with a black-box view of the inputs and outputs, then uses UML diagrams (use-case, sequence, activity and class diagrams) and an entity relationship diagram to describe how the system behaves and what data it keeps. It ends with the system architecture, the network topology as built in Containerlab, and the addressing, policy, feature and response tables that the implementation follows.

4.2 Black-Box

Figure 4.1 shows the system as a black box with three external actors. Network users (admin, staff and guest) send their 802.1X credentials and their normal traffic into the system. In return they get access to the VLAN for their role, or they are refused if the login fails. The simulated attacker sends attack traffic: a DoS flood, a port scan, brute-force logins, a large insider download or off-hours activity. What comes back to the attacker is a blocked source address, a move to the quarantine VLAN or a disabled account.

The network administrator provides the user accounts, the VLAN policy and the normal traffic run used to train the model, and can release a host that was blocked or quarantined. In return the administrator receives the anomaly scores, the alerts and the actions taken, together with the time each response took, through the Zabbix dashboard and the incident log.

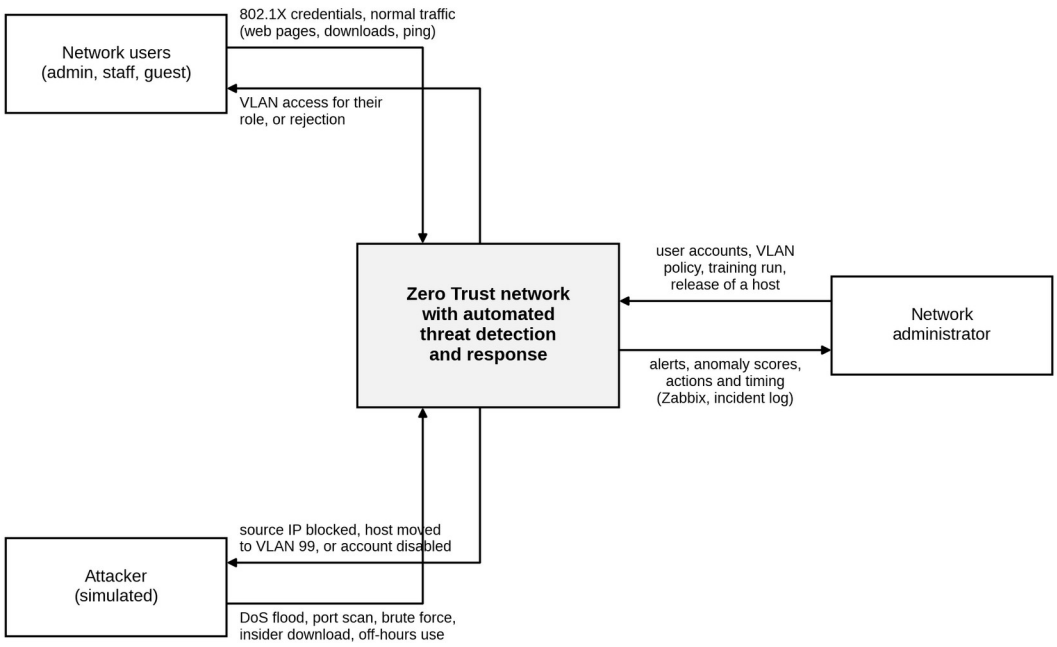


Figure 4.1: Black-box view of the system

4.3 System Analysis

4.3.1 Use-Case Diagram

Figure 4.2 shows the use cases of the system and its three actors. The network user logs in with 802.1X and then accesses the resources allowed for their role; accessing resources always includes logging in. The attacker generates attack traffic, which passes through the same network as normal traffic. The network administrator manages users and VLAN policy, trains the Isolation Forest model, monitors network traffic, views alerts and scores, and releases or re-enables hosts that the system has acted on.

Monitoring network traffic includes scoring each session with the Isolation Forest, and scoring includes extracting the session features. When the score is above the threshold, the Respond to anomaly use case extends scoring. Responding takes four specific forms: blocking the source IP, quarantining the host in VLAN 99, disabling the account and raising an alert. FreeRADIUS, the switches and Zabbix are treated as parts of the system rather than as actors, because they were built and configured as part of this project.

The diagram uses standard UML notation, which is also explained in the key under the figure. A dashed arrow marked «include» points to a use case that always runs as part of another one, a dashed arrow marked «extend» shows a use case that runs only when a condition is met, and a solid arrow with a hollow head shows that one use case is a specific kind of another.

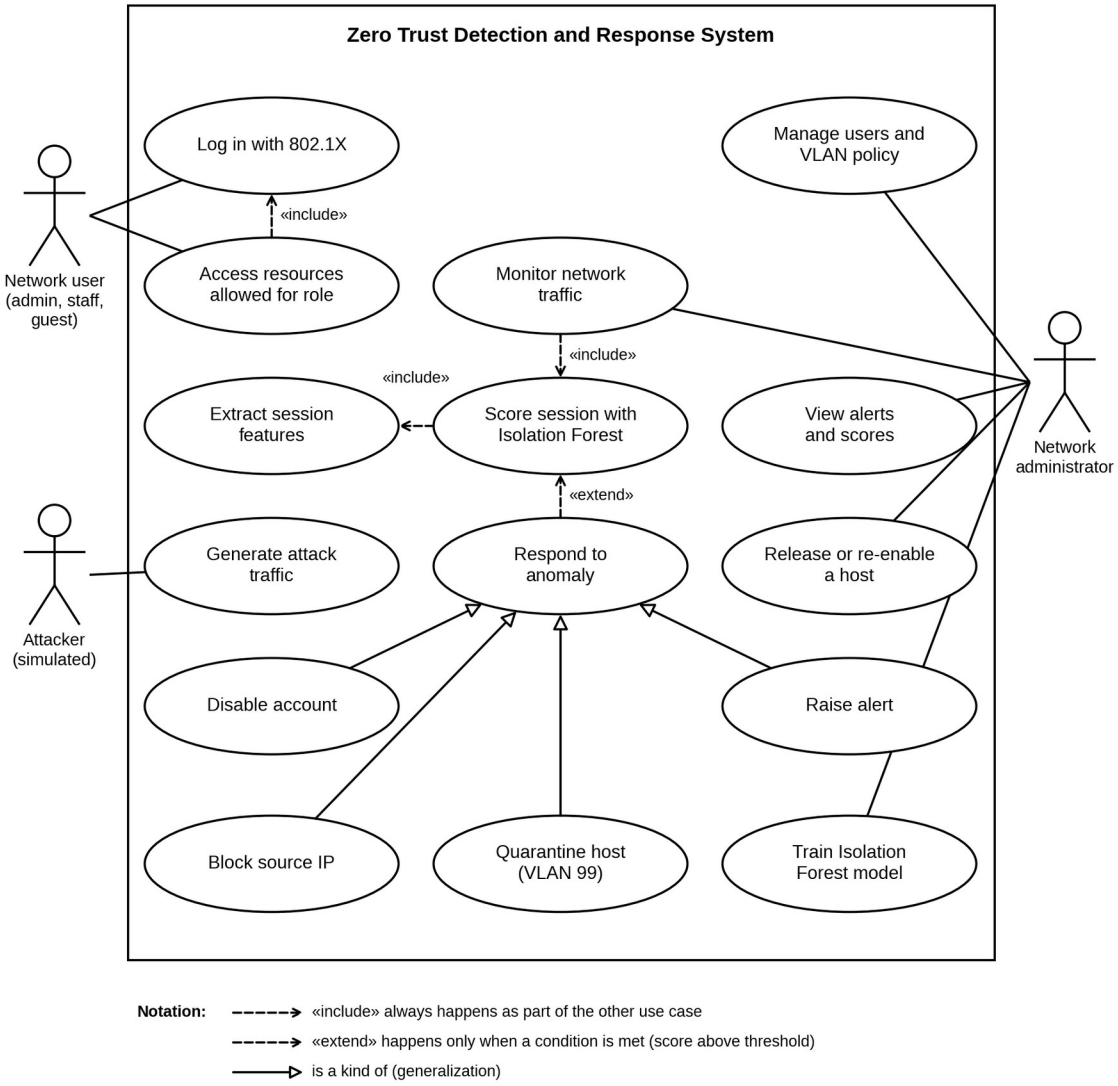


Figure 4.2: Use-case diagram

4.3.2 Sequence Diagram

Figure 4.3 shows the messages exchanged when one attack window is detected. The attacker on guest-pc sends packets through acc1, which copies them to the SIEM host on Ethernet6 (messages 1 and 2). When the ten-second window closes, the collector extracts the eight features, asks the

Isolation Forest for a score and pushes the score to Zabbix (messages 3 to 6). Scores are pushed for every window, normal or not, so the dashboard always shows the current state of each host.

If the score is above the threshold, or the window has five or more failed logins, the collector passes the record to the response engine, which classifies the attack (messages 7 and 8). The inner alt fragment shows the three possible paths. For a DoS attack the response engine adds a deny line for the source address to ZT_POLICY on the core switch, using one of the sequence numbers 71 to 79 that are reserved for this (messages 9 and 10). For a port scan, a brute force or insider data theft, it writes a rule for the user into FreeRADIUS and bounces the user’s port on acc1 (messages 11 to 14). The rule sends the user to VLAN 99 for a quarantine, or rejects the login for a disabled account. This method was chosen because 802.1X assigns the VLAN at login, so changing the VLAN of the port on the switch would be overwritten at the next login. The lab has no DHCP server, so after a quarantine the engine also gives the PC its quarantine address, for example 10.10.99.30 for guest-pc (message 15). For off-hours or unclassified anomalies the network is not changed (message 16). In every case the alert, the action and the response time are sent to Zabbix and written to the incident log (messages 17 and 18).

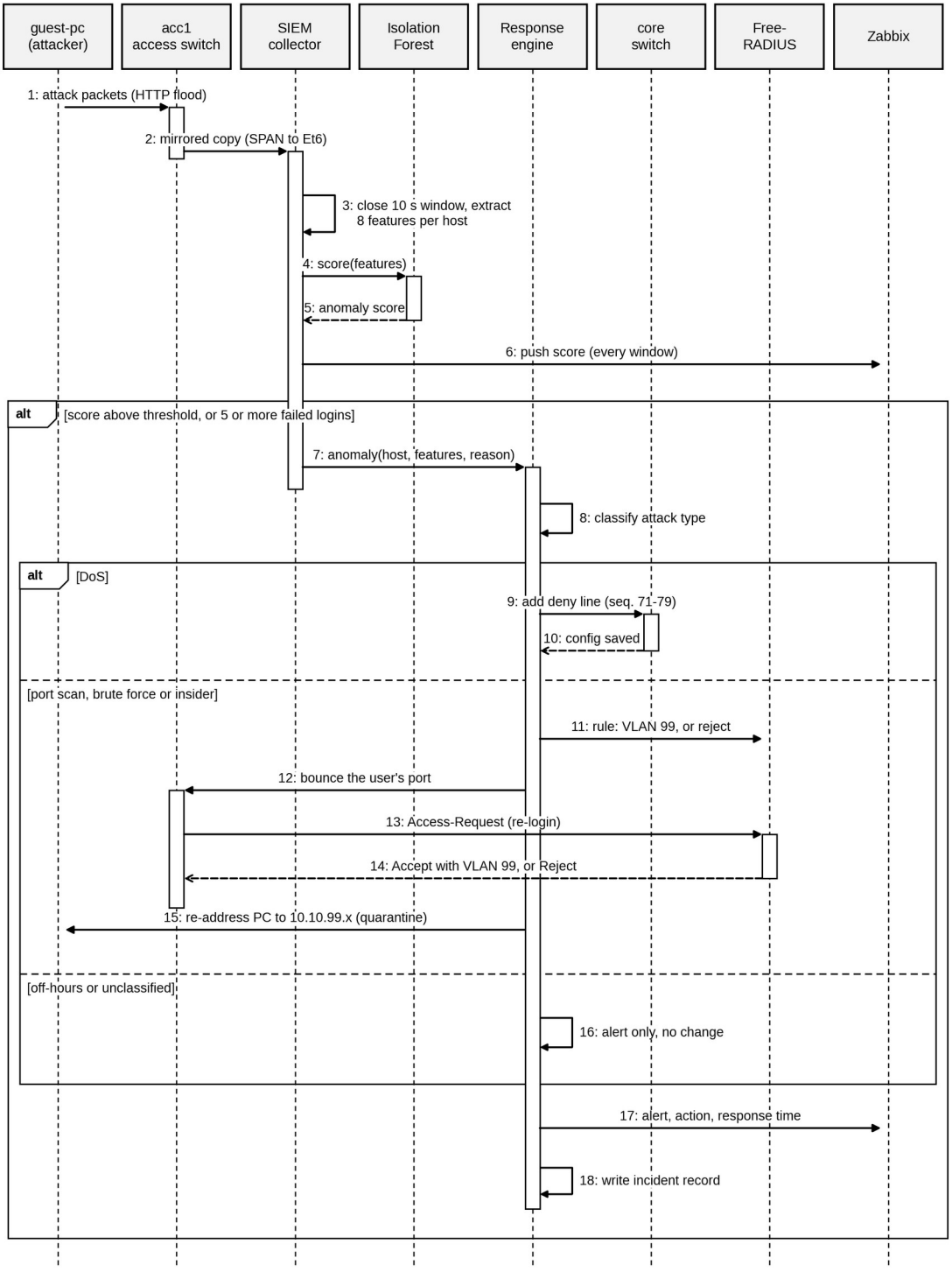


Figure 4.3: Sequence diagram for one anomalous window

4.3.3 Activity Diagram

The activity diagram in Figure 4.4 shows the loop that runs on the SIEM host. Each pass starts with two activities in parallel: capturing the mirrored packets for ten seconds and reading new Access-Reject lines from the FreeRADIUS log. When both have finished, the packets are grouped by host, the eight features are computed, the features are scaled and the Isolation Forest returns a score, which is pushed to Zabbix.

A window is flagged when its score is above the threshold or when it contains five or more failed logins. If neither is true, the loop moves on to the next window, so normal traffic is never interrupted. Otherwise the attack type is classified and one of four branches runs. A DoS adds a deny line for the source IP on the core switch; a port scan or brute force moves the user to VLAN 99 and bounces the port; insider data theft disables the account by rejecting the user’s login and bouncing the port; and off-hours or unclassified anomalies raise an alert without changing access. The branches then merge, the incident is recorded and sent to Zabbix with its response time, and the loop continues until it is stopped.

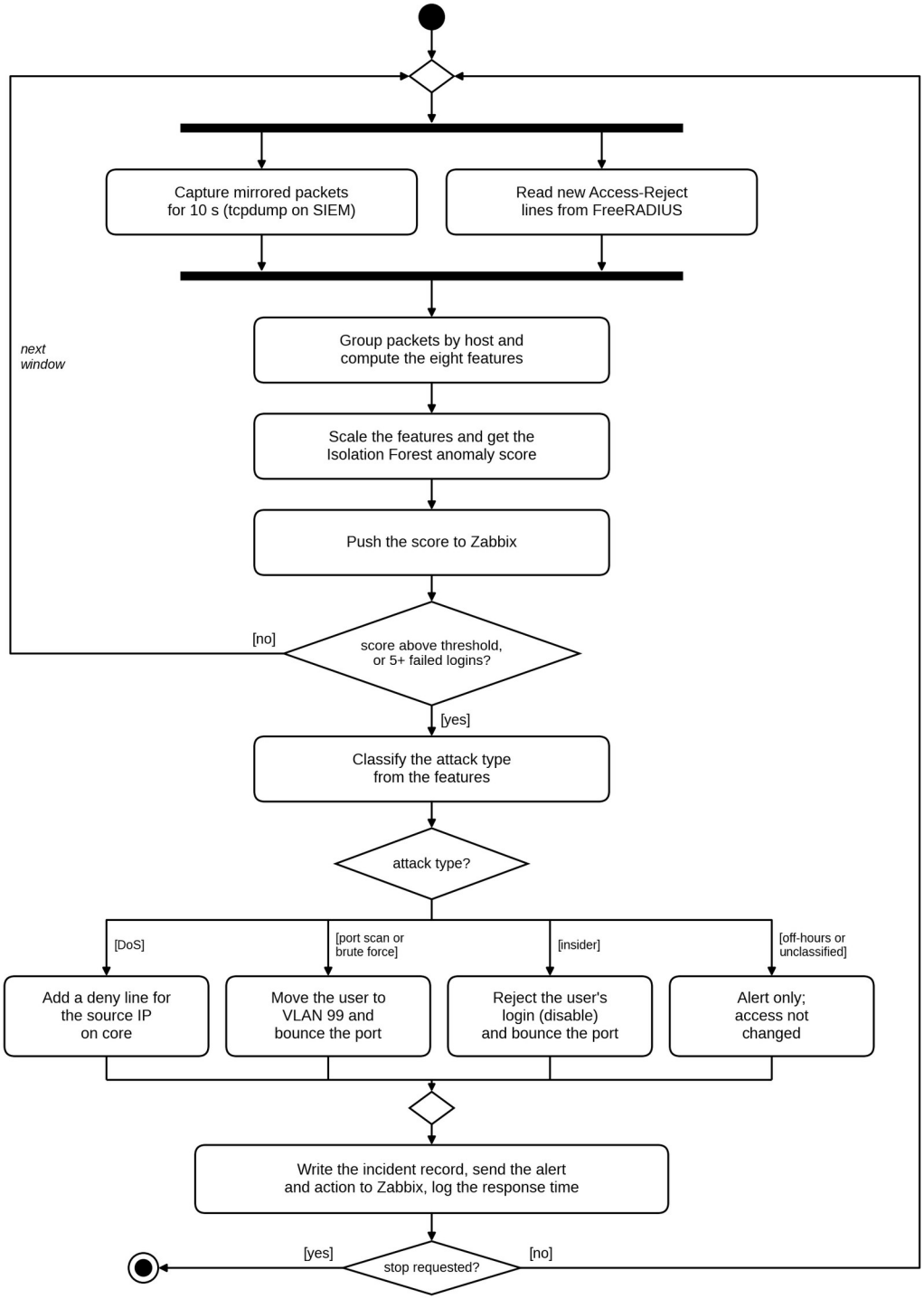


Figure 4.4: Activity diagram of the detection and response loop

4.3.4 Entity Relationship Diagram (ERD)

Figure 4.5 shows the data that the system stores and how it is related, using crow's foot notation. A VLAN is assigned to many user accounts and contains many hosts. A user account makes many

authentication events, one for each RADIUS accept or reject, and logs in on hosts. Each host generates one session window every ten seconds. SESSION_WINDOW holds the eight features, the anomaly score and, for evaluation only, the label. A session window raises at most one alert, and each alert triggers one or more response actions. RESPONSE_ACTION records what was done (block, quarantine, disable or alert only), its target, the time it took and when the host was released.

Through its target, a quarantine action changes the user's VLAN to 99 in FreeRADIUS, which changes the VLAN of the user's host at the next login. This is where the detection layer feeds back into access control. In the implementation the session windows are stored in CSV files, the alerts and actions in a JSON incident log, and the users and VLANs in the FreeRADIUS users file and the switch configuration.

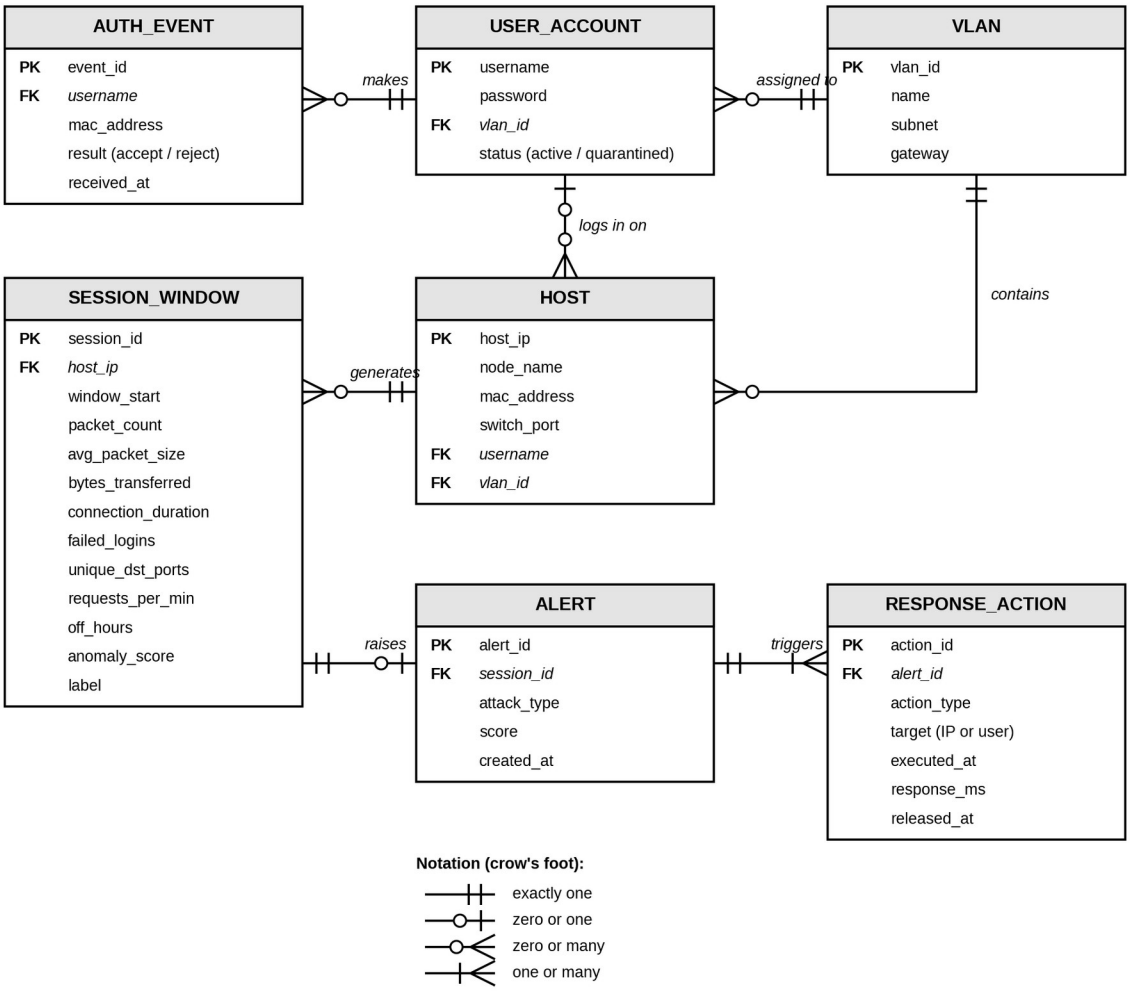


Figure 4.5: Entity relationship diagram

4.3.5 Class Diagram

Figure 4.6 shows the main Python classes and modules and the files that hold them. The zt_collector module captures the mirrored packets, reads them and builds one row of features per host per window, and it uses a RadiusRejectParser object to turn FreeRADIUS log lines into failed-login events. AnomalyDetector in zt_model.py wraps the scikit-learn StandardScaler and an IsolationForest of 200 trees, with methods to fit on rows, score them, tune the threshold on a normal run and save or load the trained model. The detect() function makes the final decision for each window: it flags the window if the model flags it or if the failed-login rule fires, and records which of the two did.

Three classes in zt_respond.py handle the response. AlertBuilder classifies a flagged window by checking its features from the most to the least specific and builds the alert record. NetworkActions produces the exact switch and RADIUS commands for each action; it changes nothing itself, so it can be tested offline. ResponseEngine applies those commands over SSH with Netmiko, keeps the SSH sessions open so that an action takes about a tenth of a second, remembers what it has already done so that a host is never acted on twice, and records every incident. The zt_zbx_push module sends scores, alerts and timings to Zabbix on TCP port 10051. PipelineOrchestrator in zt_pipeline.py owns the detector and the response engine and runs the loop shown in Figure 4.4.

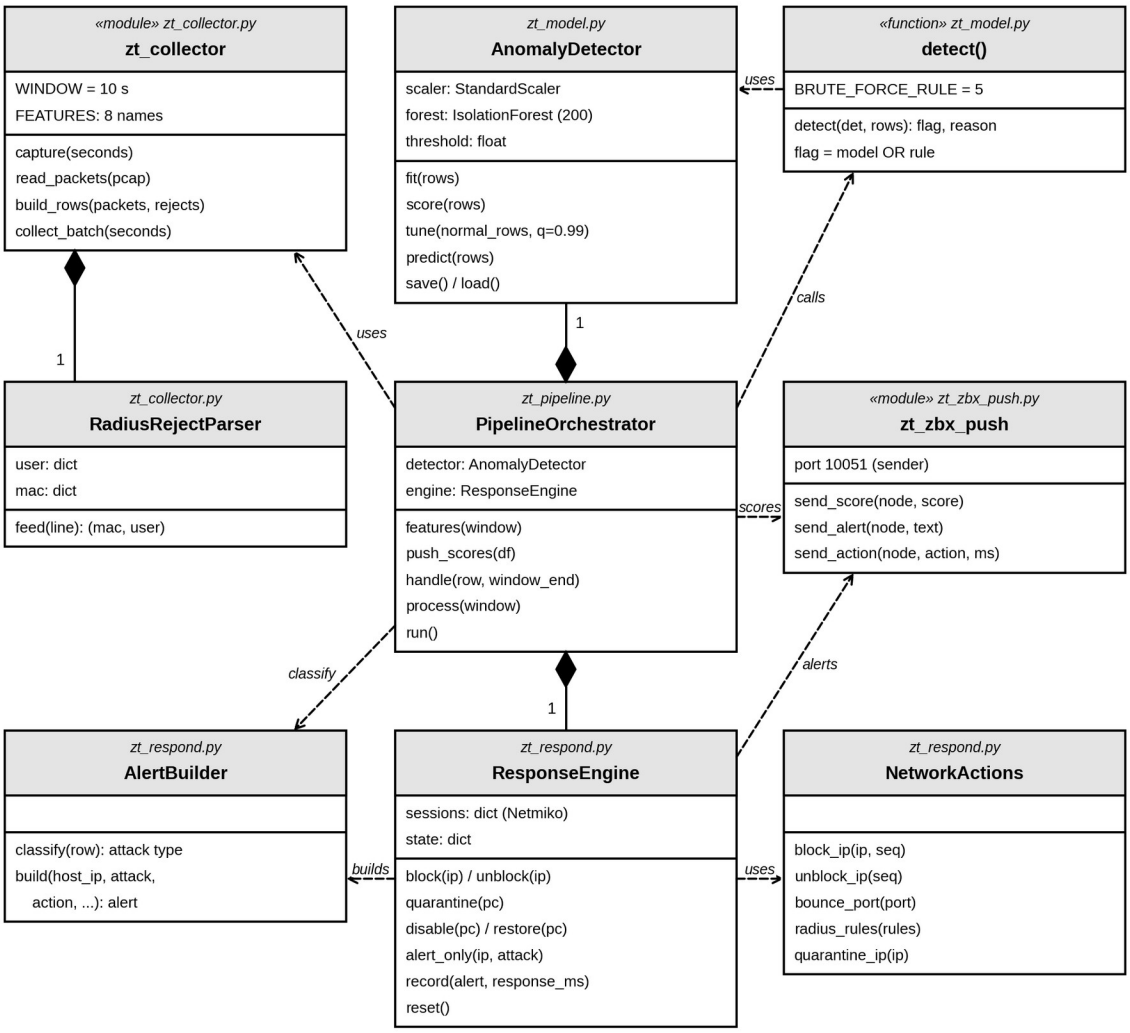


Figure 4.6: Class diagram

4.4 System Design

4.4.1 System Architecture Diagram

Figure 4.7 shows the system architecture. The network side has four layers. At the top, the edge firewall connects the lab to the internet through NAT and separates it from the DMZ web server; its default forwarding policy is DROP and only the traffic described in Section 4.4.3 is allowed. The core switch is the default gateway for VLANs 10, 20, 30 and 99, routes between them and applies the ZT_POLICY ACL. The access switch acc1 authenticates the three users with 802.1X

and copies their traffic to the SIEM host, and FreeRADIUS places each user in their VLAN after login.

The security services are in VLAN 10. FreeRADIUS answers the 802.1X requests from acc1, and the SIEM host runs the pipeline made up of the collector, the Isolation Forest, the classifier and the response engine. The response engine changes the network in two places: the ACL on the core switch for a block, and the user's rule in FreeRADIUS together with a port bounce on acc1 for a quarantine or a disabled account. Zabbix runs on the host machine and receives the scores, the alerts and the health data of every node.

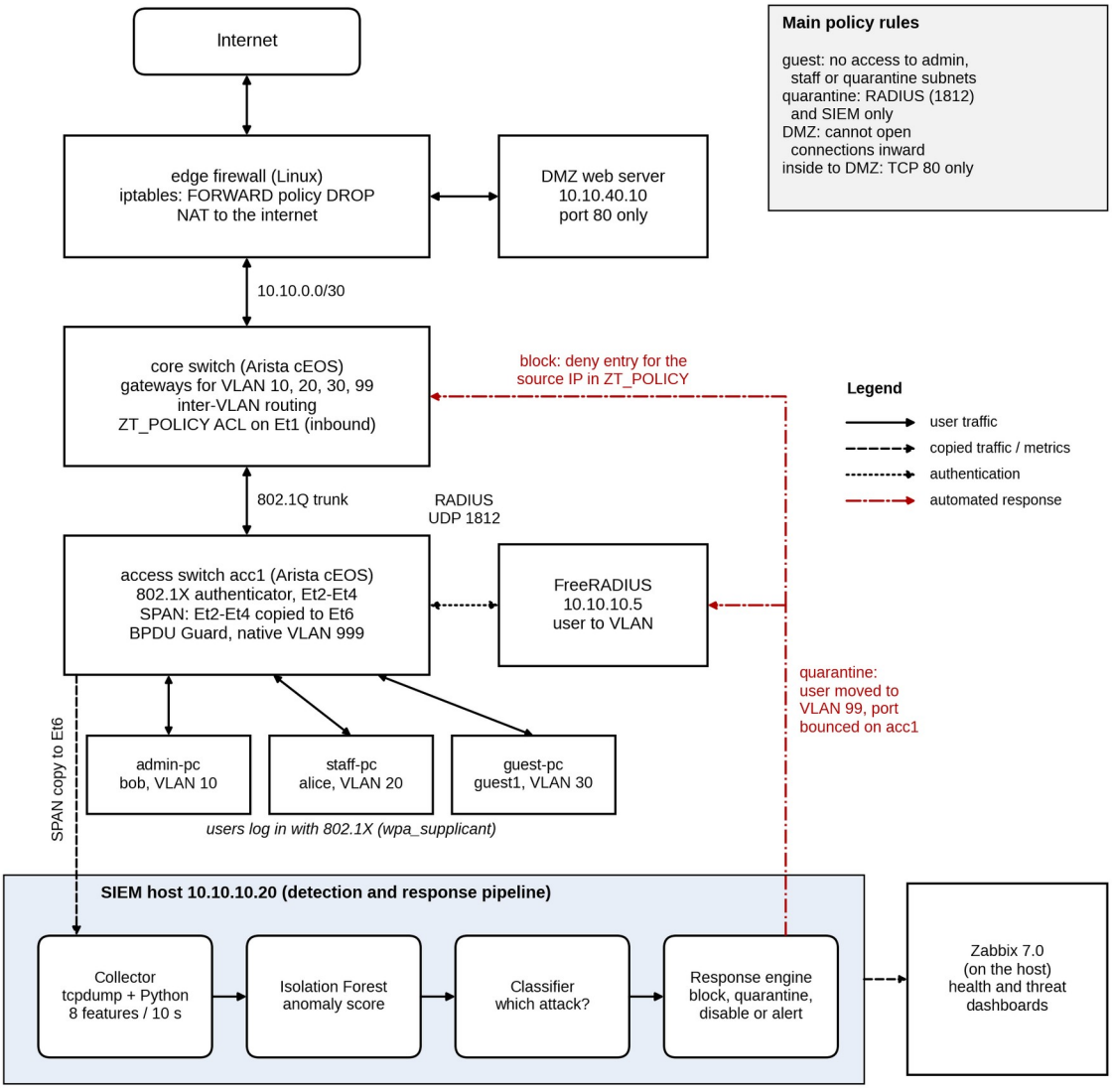


Figure 4.7: System architecture

4.4.2 Network Topology

Figure 4.8 shows the topology as built in Containerlab, with the interface names and addresses used in the configuration files. The core switch connects to the edge firewall over a routed /30 link on Ethernet3, and to acc1 over an 802.1Q trunk on Ethernet1. The trunk carries only VLANs 10, 20, 30 and 99 and uses an unused native VLAN (999) to prevent VLAN hopping. On acc1, Ethernet2 to Ethernet4 are 802.1X ports for the three PCs, with PortFast and BPDU Guard enabled. Ethernet5 and Ethernet6 are fixed VLAN 10 ports for FreeRADIUS and the SIEM host, and Ethernet6 is also the SPAN destination. Table 4.1 gives the addressing plan.

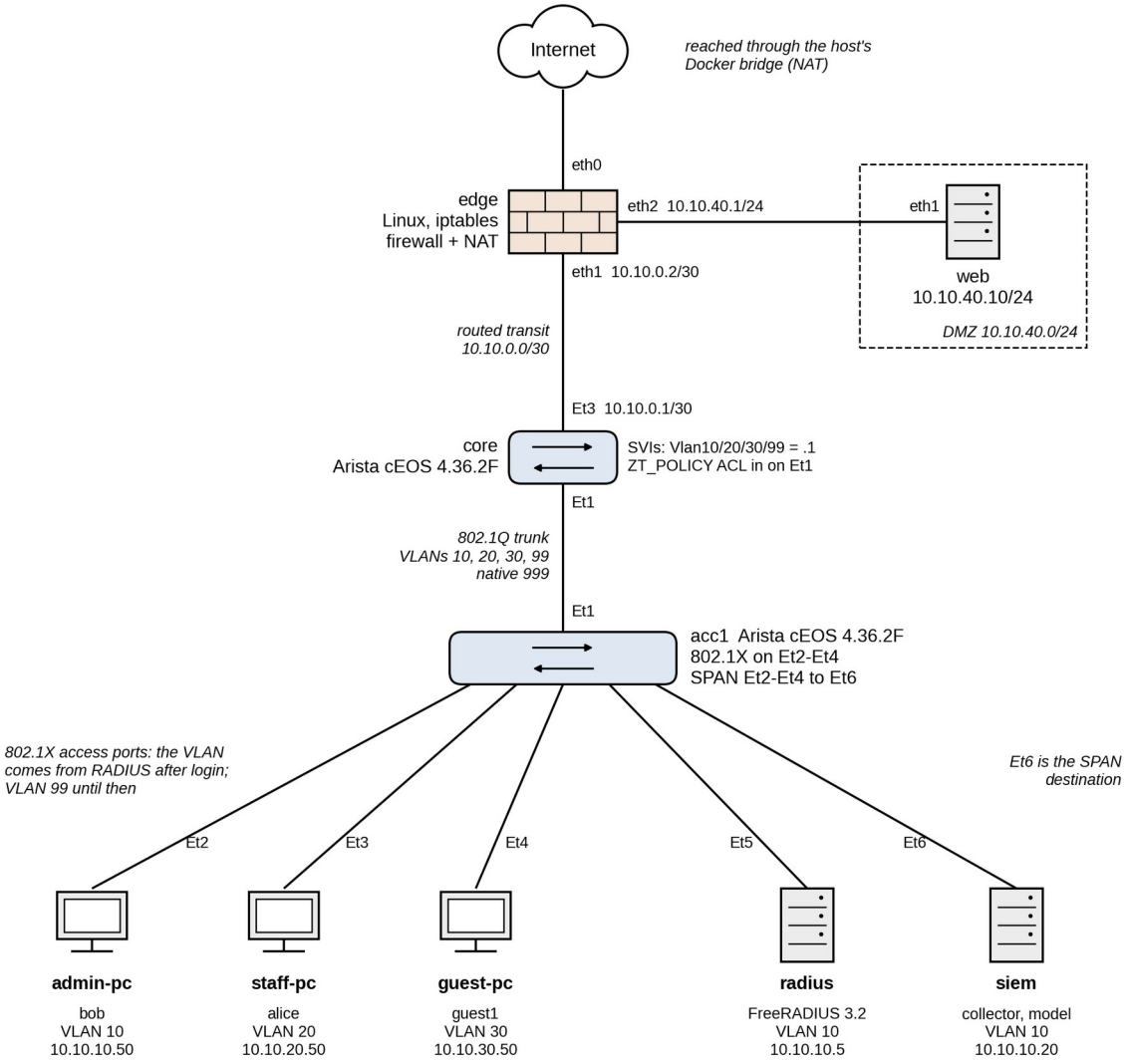


Figure 4.8: Network topology as built in Containerlab

VLAN	Name	Subnet	Gateway	Hosts
10	ADMIN	10.10.10.0/24	10.10.10.1	admin-pc .50, radius .5, siem .20, acc1 .11
20	STAFF	10.10.20.0/24	10.10.20.1	staff-pc .50
30	GUEST	10.10.30.0/24	10.10.30.1	guest-pc .50
99	QUARANTINE	10.10.99.0/24	10.10.99.1	Quarantined PCs: admin .10, staff .20, guest .30
999	NATIVE_UNUSED	None	None	Native VLAN of the trunk only
DMZ	None	10.10.40.0/24	10.10.40.1 (edge)	web .10
Transit	None	10.10.0.0/30	None	core .1, edge .2

Table 4.1: VLAN and addressing plan

4.4.3 Access Control Policy

The segment policy is enforced in two places. On the core switch, the ZT_POLICY ACL is applied inbound on Ethernet1, the trunk from acc1, so it sees all traffic from the user VLANs before it is routed. It was first planned as separate ACLs on the VLAN interfaces, but cEOS-lab does not support ACLs on VLAN interfaces, so the rules were merged into one ACL on the trunk port. Table 4.2 lists its entries. When the response engine blocks an address, it inserts a deny line for that address in the range 71 to 79, which is reserved for this between the guest rules and the final permit.

Seq.	Action	Traffic matched	Purpose
10	permit	UDP from port 68 (bootpc) to port 67 (bootps)	DHCP
20	permit	10.10.99.0/24 to 10.10.10.5, UDP 1812	A quarantined host may log in again
30	permit	10.10.99.0/24 to 10.10.10.20	A quarantined host may be investigated
40	deny	10.10.99.0/24 to any	Quarantine reaches nothing else
50	deny	10.10.30.0/24 to 10.10.10.0/24	Guest to admin blocked
60	deny	10.10.30.0/24 to 10.10.20.0/24	Guest to staff blocked
70	deny	10.10.30.0/24 to 10.10.99.0/24	Guest to quarantine blocked
80	permit	any to any	All other traffic

Table 4.2: Entries of the ZT_POLICY access list

Staff traffic to the admin VLAN is currently allowed by entry 80. Limiting staff to only the admin services they need would be a further improvement in line with least privilege.

The edge firewall adds the perimeter rules with iptables. Forwarded traffic is dropped by default, and replies to existing connections are allowed. The internal VLANs may go out to the internet and may reach the DMZ web server only on TCP port 80. The DMZ may reach the internet, but any connection it starts toward the internal network is dropped. Outgoing traffic is translated (NAT) to the edge’s external address.

4.4.4 Detection Features and Response Rules

The collector turns the traffic of each host in each ten-second window into the eight features listed in Table 4.3. Each feature was chosen because at least one of the threats in Table 2.1 changes it.

Feature	Meaning	Attack it reveals
packet_count	Packets sent or received by the host	DoS, data theft
avg_packet_size	Mean IP packet length	Data theft (large), floods (small)
bytes_transferred	Total IP bytes in the window	Insider data theft
connection_duration	Mean flow duration	Port scan (very short flows)
failed_logins	RADIUS Access-Rejects for the host's MAC address	Brute force
unique_dst_ports	Distinct destination ports the host opened	Port scan
requests_per_min	New connections started by the host, per minute	DoS, port scan
off_hours	1 if the window is outside 08:00 to 17:59	Off-hours use

Table 4.3: Features extracted for each host in each window

The response depends on the attack type, as shown in Table 4.4. The classifier checks the signs in the order of the table, from the most to the least specific, and each limit is far above normal traffic: the 99th percentile of normal windows is 540 packets, 430 kB, one destination port and 48 new connections per minute. Large downloads can also come from legitimate work, so insider data theft disables the account instead of isolating the host, and the administrator can re-enable it after checking. Work at night is only reported.

Attack type	Main signs	Action	Where it is applied
DoS	requests_per_min of 300 or more	Block source IP	Deny line (71 to 79) in ZT_POLICY on core
Port scan	unique_dst_ports of 10 or more	Quarantine host	VLAN 99 rule in FreeRADIUS and port bounce on acc1
Brute force	failed_logins of 5 or more	Quarantine host	VLAN 99 rule in FreeRADIUS and port bounce on acc1
Insider data theft	bytes_transferred of 5 MB or more	Disable account	Reject rule in FreeRADIUS and port bounce on acc1
Off-hours or unclassified	off_hours = 1, or none of the above	Alert only	Zabbix and incident log

Table 4.4: Response taken for each attack type

References

Abuhasel, K. A. (2023). A zero-trust network-based access control scheme for sustainable and resilient Industry 5.0. *IEEE Access, 11*, 116398–116409. https://doi.org/10.1109/ACCESS.2023.3325879

Ahmadi, S. (2024). Zero trust architecture in cloud networks: Application, challenges and future opportunities. *Journal of Engineering Research and Reports, 26*(2), 215–228.

Ahmadi, S. (2025). Autonomous identity-based threat segmentation for zero trust architecture. *Cyber Security and Applications, 3*, 100106.

Ali, M. L., Thakur, K., Schmeelk, S., Debello, J., & Dragos, D. (2025). Deep learning vs. machine learning for intrusion detection in computer networks: A comparative study. *Applied Sciences, 15*(4), 1903.

Al-Tamimi, S., Al-Haija, Q. A., & Alrawashdeh, K. (2024). Zero-trust architecture for securing Internet of Things (IoT) networks: A review. In *2024 5th International Conference on Communications, Information, Electronic and Energy Systems (CIEES)* (pp. 1–6). IEEE. https://doi.org/10.1109/CIEES62939.2024.10811176

Altidor, J.-B., & Talhi, C. (2024). Enhancing port scan and DDoS attack detection using genetic and machine learning algorithms. In *2024 7th Conference on Cloud and Internet of Things (CIoT)* (pp. 1–7). IEEE. https://doi.org/10.1109/CIoT63799.2024.10757005

Aramide, O. (2024). Zero-trust identity principles in next-gen networks: AI-driven continuous verification for secure digital ecosystems. *World Journal of Advanced Research and Reviews, 23*(3), 3304–3316.

Arista Networks. (n.d.). *EOS user manual*. Retrieved October 14, 2026, from https://www.arista.com/en/um-eos

Baligodugula, V. V., Ghimire, A., & Amsaad, F. (2024). An overview of secure network segmentation in connected IIoT environments. *Computing & AI Connect, 1*, Article 0004.

Bello, A. J., Diyan, M., & Asghar, I. (2025). Zero trust implementation for legacy systems using dynamic microsegmentation, role-based access control (RBAC), and attribute-based access control (ABAC). In *2025 4th International Conference on Computing and Information Technology (ICCIT)* (pp. 181–189). IEEE. https://doi.org/10.1109/ICCIT63348.2025.10989392

Bradatsch, L., & Kargl, F. (2024). Integration of security service functions into network-level access control. *IEEE Access, 12*, 197783–197815. https://doi.org/10.1109/ACCESS.2024.3522575

Bredesen, R., & Mujeye, S. (2025). Network segmentation security with the implementation of threats. In *Proceedings of the 2025 8th International Conference on Software Engineering and Information Management (ICSIM '25)* (pp. 137–141). Association for Computing Machinery.

Byers, K. (n.d.). *Netmiko* [Computer software]. GitHub. Retrieved October 14, 2026, from https://github.com/ktbyers/netmiko

Chang, P., & Xu, G. (2026). Design and implementation of a zero trust access control model driven by session lifecycle. *IEEE Access, 14*, 33819–33827. https://doi.org/10.1109/ACCESS.2026.3669103

Chen, G., Hu, Z., Qu, Y., & Jin, D. (2025). VT-IO: A virtual time system enabling high-fidelity container-based network emulation for I/O intensive applications. *ACM Transactions on Modeling and Computer Simulation, 35*(1), 1–25. https://doi.org/10.1145/3635307

Choi, B. (2024). Python network automation labs: SSH in action, paramiko and netmiko labs. In *Introduction to Python network automation volume II* (pp. 121–227). Apress. https://doi.org/10.1007/979-8-8688-0391-8_3

Cisco Networking Academy. (n.d.). *Cisco Packet Tracer*. Retrieved October 14, 2026, from https://www.netacad.com/cisco-packet-tracer

Containerlab. (n.d.). *Arista cEOS*. Retrieved October 14, 2026, from https://containerlab.dev/manual/kinds/ceos/

Dube, R. (2026). A taxonomy of segmentation in network security. *IEEE Access, 14*, 16921–16935. https://doi.org/10.1109/ACCESS.2026.3658250

Falowo, O. I., & Bou Abdo, J. (2026). Empirical study on automation, AI trust, and framework readiness in cybersecurity incident response. *Algorithms, 19*(1), 62.

FreeRADIUS. (n.d.). *FreeRADIUS documentation*. Retrieved October 14, 2026, from https://www.freeradius.org/documentation/

Fuhnwi, G. S., Adedoyin, V., & Agbaje, J. O. (2023). An empirical Internet Protocol network intrusion detection using Isolation Forest and One-Class Support Vector Machines.

International Journal of Advanced Computer Science and Applications, 14(8). https://doi.org/10.14569/IJACSA.2023.0140801

Hidayat, I., Ali, M. Z., & Arshad, A. (2023). Machine learning-based intrusion detection system: An experimental comparison. *Journal of Computational and Cognitive Engineering, 2*(2), 88–97.

Islam, M. D. Z., & Dhanekula, A. (2023). Measuring the security impact of zero trust access controls: A mixed-methods study of identity-based policies (Cisco ISE + AD) and incident reduction. *American Journal of Data Science and Analytics, 4*(6), 1–42.

Jones, R. K. (2025). AI automated incident response and threat mitigation using AI. In *Revolutionizing cybersecurity with deep learning and large language models* (pp. 201–236). IGI Global Scientific Publishing.

Kukartsev, V., Kravtsov, K., Stefanenko, O., Podanyov, N., & Bezvorotnykh, A. (2024). Using machine learning techniques to simulate network intrusion detection. In *2024 International Conference on Intelligent Systems for Cybersecurity (ISCS)* (pp. 1–4). IEEE.

Kumar, P. V., Pittala, R. B., Jashwanth, P., Vedamsh, R., Chandrika, D. S., Reddy, Y. V., Kiran, M. A., & Reddy, G. S. (2025). Automated threat detection and response using security orchestration, automation, and response (SOAR) and endpoint detection and response. In *2025 OITS International Conference on Information Technology (OCIT)* (pp. 498–502). IEEE. https://doi.org/10.1109/OCIT66168.2025.11400041

Lakshmi, M., Rajavikram, G., Dattatreya, V., Jyothi, B. S., Patil, S., & Bhavsingh, M. (2024). Evaluating the isolation forest method for anomaly detection in software-defined networking security. *Journal of Electrical Systems, 19*(4), 279–297. https://doi.org/10.52783/jes.639

Mensah, F. (2024). Zero trust architecture: A comprehensive review of principles, implementation strategies, and future directions in enterprise cybersecurity. *International Journal of Academic and Industrial Research Innovations, 10*, 339–346.

Moomtaheen, F., Bagui, S. S., Bagui, S. C., & Mink, D. (2024). Extended isolation forest for intrusion detection in Zeek data. *Information, 15*(7), 404. https://doi.org/10.3390/info15070404

Nam, S.-W., Yu, G.-M., Jang, Y.-S., Kim, J.-S., Kim, J.-M., & Kim, M.-S. (2026). Trust quantification on zero trust architecture: A survey. *IEEE Access, 14*, 105289–105313. https://doi.org/10.1109/ACCESS.2026.3712376

Rauf, U., Wei, Z., & Mohsen, F. (2023). Employee Watcher: A machine learning-based hybrid insider threat detection framework. In *2023 7th Cyber Security in Networking Conference (CSNet)* (pp. 39–45). IEEE. https://doi.org/10.1109/CSNet59123.2023.10339777

Rautaray, A. (2025). Machine learning techniques applied to intrusion detection systems. *Power System Protection and Control, 53*(1), 17–29.

scikit-learn developers. (n.d.). *IsolationForest*. Retrieved October 14, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

Syzranova, S., Pleva, M., & Harahus, M. (2026). Benign-only training for network intrusion detection: A comparative study of Isolation Forest, One-Class SVM, and autoencoders. In *2026 36th International Conference Radioelektronika (RADIOELEKTRONIKA)* (pp. 1–6). IEEE. https://doi.org/10.1109/RADIOELEKTRONIKA71189.2026.11726544

Vadisetty, R., & Polamarasetti, A. (2024). Enhancing intrusion detection systems with deep learning and machine learning algorithms for real-time threat classification. In *2024 Asian Conference on Intelligent Technologies (ACOIT)* (pp. 1–6). IEEE.

Zabbix. (n.d.). *Zabbix 7.0 manual*. Retrieved October 14, 2026, from https://www.zabbix.com/documentation/7.0/en/manual